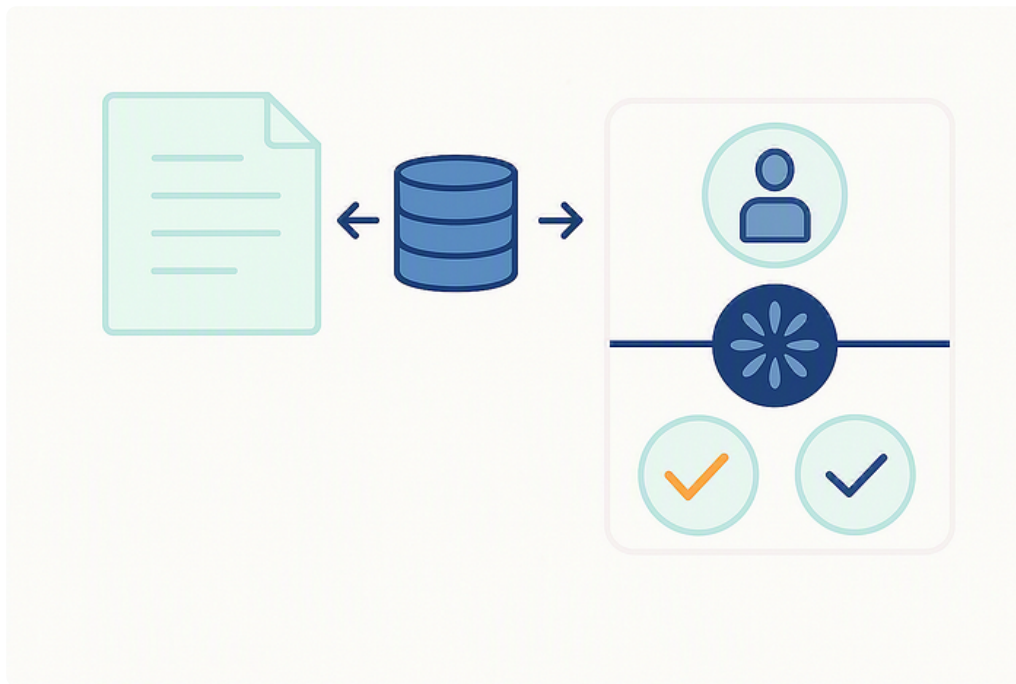




Part 4: Extracting Ontologies from Sources

A series of articles on semantic foundations for goal-directed intelligence: Table of Content

Yuan An, PhD

Published June 30, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

What you will learn	3
1. Building ontologies by experts and by machine	3
2. Upper ontologies: the expert-built foundation	4
3. The invariants	6
Recall-first	6
Provenance everywhere	6
Grounding by kind	6
Universals, not particulars	6
Scope before you extract	6
4. Stage 0: Scope	7
5. Stage 1: Surface candidates	8
6. Stage 1b: Sort by kind (the type/instance gate)	8
7. Stage 2: Name the vocabulary	9
8. Stage 2b: Rank by salience	10
9. Stage 3: Synthesize structure	10
Anchor before you coin	11
Relations get a real domain and range	11
10. Stage 4: Agentic RITE review	12
11. SCIMA example: extracting v0.6 from the procedures corpus	13
Anchoring v0.6 to verified ontologies	16
Key takeaways	17
Further reading	17

A series of articles on semantic foundations for goal-directed intelligence:
[Table of Content](https://freedium-mirror.cfd/https://medium.com/@anyuanay/ontology-and-knowledge-graphs-and-ai-agents-ce172be02ea1) (https://freedium-mirror.cfd/https://medium.com/@anyuanay/ontology-and-knowledge-graphs-and-ai-agents-ce172be02ea1)

What you will learn

- An ontology can be built **manually by experts** or **(semi-)automatically by machine**.
- Know the major expert-built **upper ontologies** and verified **domain ontologies**, and the rule that follows: **anchor before you coin**.
- Walk the extraction pipeline as it is actually built, seven stages: **Scope, Surface, Sort by kind, Name, Rank by salience, Synthesize structure, and Review**.
- See why an ontology is a **conceptualization**: it holds the **universals** of a domain (its classes and relationships), not the **individual facts** a text happens to mention.
- Hold the invariants that run the length of the pipeline: **recall-first, provenance everywhere, grounding by kind, universals not particulars, and scope before you extract**.
- See how an LLM is used where it is strong (scoping, sorting, naming, synthesizing structure) but always **fenced by grounding**, and how an agent drives the RITE review (Refine, Inspect, Test, Extend) under human oversight.
- Run the pipeline over a SCIMA corpus, anchor the result to verified upper and domain ontologies, and watch SCIMA-OWL grow to **v0.6**.

1. Building ontologies by experts and by machine

The first three articles treated SCIMA-OWL as something we authored by hand. There are two complementary ways an ontology comes to exist.

- **Manual authoring by experts.** Ontologists and domain experts decide the categories and the axioms deliberately and justify each commitment. The top of every serious ontology, the upper (foundational) layer, is built this way and should be: an ill-judged top-level distinction is felt everywhere below it.
- **(Semi-)automatic construction by machine.** Algorithms and language models read sources and propose ontology elements, which a human (or an agent under human oversight) reviews and ratifies.

Extraction is harder than it sounds, because text does not hand you an ontology. It hands you a stream of noisy mentions: synonyms for the same thing (IC and incident commander), the same word for different things, implicit hierarchies stated nowhere, relations buried in verbs, and, above all, a flood of particulars. A textbook or a manual is mostly worked examples and exercises: specific numbers, dates, money amounts, named people. Those are individual facts, not concepts. An ontology is a **conceptualization** of a domain: it captures the general kinds and relationships. Keeping universals and particulars apart is the difference between an ontology and a data dump, and it is the single thing a naive extractor gets most wrong.

That is what a [pipeline](https://github.com/anyuanay/AI_agent_applications/tree/main/ontology_KG_extraction_skills/ontology_extraction_skills) (https://github.com/anyuanay/AI_agent_applications/tree/main/ontology_KG_extraction_skills/ontology_extraction_skills) we developed for ontology extraction: a sequence of stages, each with one job. The pipeline is framed top-down by a **scope**, fed by high-recall surfacing, and then narrowed in deliberate steps, sorting out particulars, naming, ranking by importance, and only then committing to structure, under review.

2. Upper ontologies: the expert-built foundation

An **upper** (or **foundational**) ontology fixes the most general categories that cut across every domain: object versus process, continuant versus occurrent, role, quality, the part-of and participation relations. These are built and validated by experts, not extracted from a corpus, and you should not try to learn them. You reuse one. Reusing a foundation buys **quality you could not bootstrap** and **interoperability**: two ontologies extracted independently still meet wherever they both anchor into the same foundation.

Upper ontology	Maintainer / standard	In a sentence
BFO (Basic Formal Ontology)	ISO/IEC 21838-2	A small realist top used widely in biomedicine and defense; the de facto ISO standard.
DOLCE	ISTC-CNR (Laboratory for Applied Ontology)	A cognitively-oriented foundation built around how humans conceptualize the world.
SUMO	IEEE-originated, open	A large upper-plus-mid ontology with a full first-order axiomatization.
GFO	Onto-Med, University of Leipzig	A foundation designed to support both objects and processes.
UFO	NEMO, UFES (drives OntoUML)	A foundation focused on conceptual modeling: kinds, roles, and phases.
Cyc / OpenCyc	Cycorp	A vast hand-curated commonsense ontology and knowledge base.
gist	Semantic Arts	A minimalist business-oriented upper ontology meant to be small and pragmatic.

Between the foundation and your freshly-extracted classes sit reusable **domain and utility ontologies**: expert-built, validated vocabularies for recurring concerns. SCIMA, a smart-city system, anchors into these rather than reinventing sensors, time, space, or provenance:

Domain / utility ontology	Standard body	Supplies
PROV-O	W3C	Provenance: <code>prov:Agent</code> , <code>prov:Entity</code> , <code>prov:wasDerivedFrom</code> .
SOSA / SSN	W3C / OGC	Sensors and observations: <code>sosa:Sensor</code> , <code>sosa:Observation</code> .
OWL-Time	W3C	Temporal entities and relations.
GeoSPARQL	OGC	Spatial features and topological relations: <code>geo:Feature</code> .
SAREF / SAREF4CITY	ETSI	Smart-city devices, assets, and infrastructure.
Empathj · MOAC	Community	Emergency management and crisis response.

This sets the rule that governs structure later: **anchor before you coin**. When the pipeline needs a parent for an extracted class, it reaches for an existing verified class (in SCIMA itself, then a domain ontology, then the upper ontology) before it coins a new one. Coining is the last resort, used only when no verified foundation offers the right home.

3. The invariants

Before the stages, the rules they all obey. These invariants are what make the difference between a pipeline you can trust and a clever prompt you cannot.

Recall-first

Nothing the corpus supports is silently dropped. Pruning is deferred and, when it happens, a rejected or parked element is recorded as feedback, not deleted. Recall-first is satisfied by never losing anything, not by admitting everything: later stages move the long tail to a feedback set rather than into the ontology.

Provenance everywhere

Every record carries where it came from: the extractor that surfaced it, the exact character offsets of every literal occurrence, and, downstream, how each decision was made (which signal proposed a parent, why an element was sorted out). Provenance is what lets a reviewer answer the only question that matters: why is this here?

Grounding by kind

What it means to justify an element depends on its kind. An extracted concept is grounded by a mention. A coined parent is grounded by its children. A relationship is grounded by a sane domain and range. An axiom is grounded by a reasoner finding no contradiction.

Universals, not particulars

The ontology (the T-Box) holds general kinds and relationships. Particulars (specific numbers, dates, people, worked-example values) are individuals: they belong to the A-Box, not to the ontology. The pipeline separates the two explicitly rather than letting instances pollute the schema.

Scope before you extract

A bottom-up extractor has no idea what the document is about or how abstract the ontology should be. A top-down scope (the domain, its topics, and the questions the ontology must answer) frames and bounds the whole pipeline, and tells the later stages what to keep out.

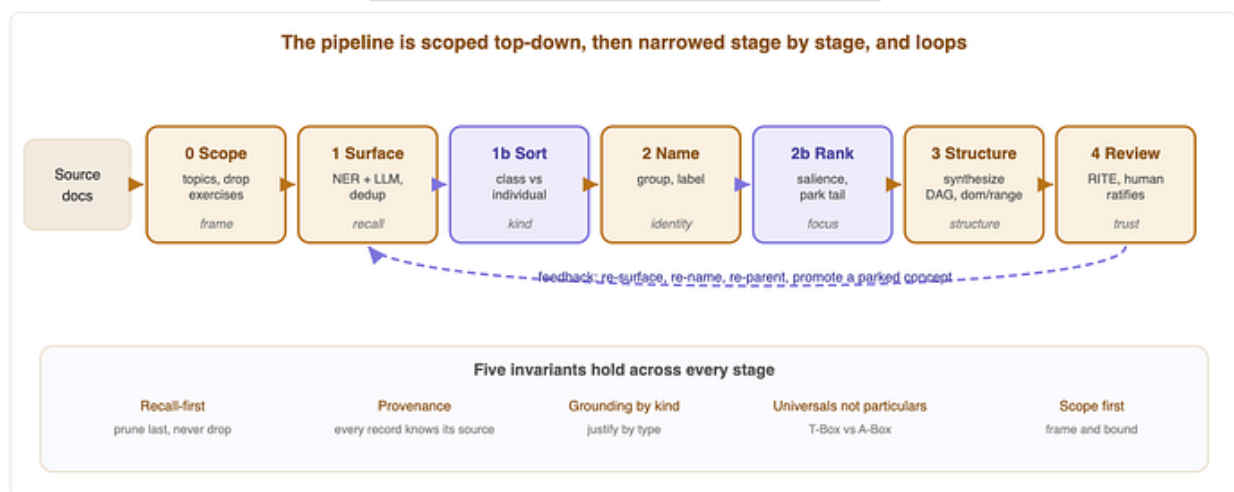


Figure 1. The seven-stage pipeline. Amber stages narrow toward the ontology; the two purple stages (Sort by kind, Rank by salience) are the gates that keep particulars and the long tail out. Scope frames everything up front; Review ratifies at the end; the purple feedback lane carries rejected, parked, and unresolved items back upstream for the next pass.

4. Stage 0: Scope

Purpose. Frame the ontology before extracting it. The output is a scope artifact: a domain statement, the topics, the seed concepts the author themselves defined, the general relations, competency questions the ontology should answer, and an explicit out-of-scope note naming the instance and example kinds to keep out.

A well-written source carries its own scaffold, and a structured source carries a rich one. The stage mines it deterministically. For example, if we use a textbook as the source for extracting an ontology, the pipeline can mine the **topics** from the table of contents and section headings, **objectives** from the "you will be able to" lists, and **defined terms** from explicit cues ("... are called the counting numbers") and bold key terms. An LLM then summarizes this skeleton into the domain statement, general key concepts, competency questions, and the out-of-scope note. The LLM only ever sees the grounded skeleton, so it raises abstraction without inventing a domain.

Stage 0 also does the single most effective thing for an instance-dense source: it **drops the content that unlikely contributes ontological terms such as the exercise and worked-example sections in a textbook source.**

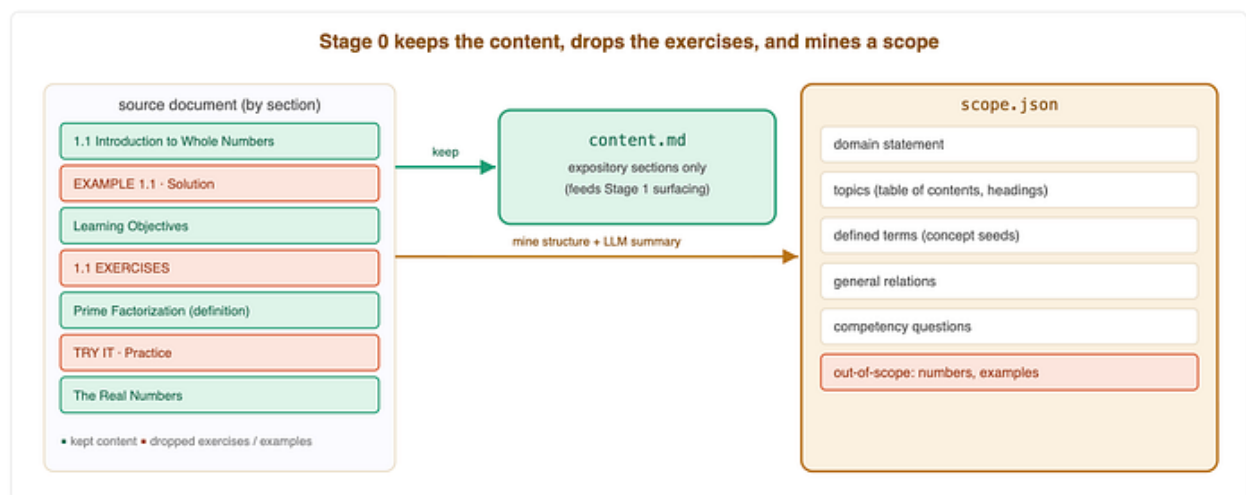


Figure 2. Stage 0 segments the source by section and drops the content (red) that does not contribute ontological terms so their specific values never reach surfacing, writing a content-only file (green). In parallel it mines the structures, objectives, and definitions and has an LLM summarize them into a scope: domain statement, topics, seed concepts, competency questions, and an out-of-scope note.

5. Stage 1: Surface candidates

Purpose. Maximize recall of terms that could become ontology elements, with zero structural commitment. The input is the content-only text from Stage 0; the output is a noisy bag of candidate mentions, each a concept-mention or relation-mention tagged with its provenance.

The mechanism is the union of two complementary extractors, then a dedup. A **cheap extractor** (NER plus unsupervised term and predicate extraction, using a library such as spaCy) is the recall floor and the hallucination-free anchor: every candidate it emits is a literal span copied out of the text. An **LLM extractor** adds the complementary lift, the multi-word, implicit, and relational candidates the cheap methods miss. The model is prompted for terms only; any structure it volunteers is discarded here. A lexical and morphological **dedup** then merges string variants so the union becomes a set. Whatever the LLM adds beyond a literal span is tagged, so later stages can give it a harder look.

6. Stage 1b: Sort by kind (the type/instance gate)

Purpose. Separate the universals from the particulars before naming. Every candidate is sorted into one of three bins: a **class** (a kind of thing, which goes on into the ontology), an **individual** (a particular such as a specific number, date, money amount, or named person, routed to an A-Box instances file for the knowledge-graph pass), or a **non-concept** (markup debris or a numbered

document label, parked as feedback). This is the highest-leverage fix for an over-large, instance-polluted ontology, and it preserves recall: particulars are routed, not deleted.

The reliable signal is **morphology**, not named-entity labels. On real text an off-the-shelf NER tagger is far too noisy to route on: it will tag `equation` as MONEY and `fraction` as PERSON, so using it to filter would throw away core concepts. Morphology is dependable: a digit (`3 years` , `49-cent stamps`) marks an individual; LaTeX or markup debris marks a non-concept; a numbered heading (`Chapter 1 Foundations`) marks a document label. The clean common nouns that remain are classes, except for the named individuals with no numeric tell (a person, a day, a place), which an LLM tiebreak catches using the Stage 0 domain statement and out-of-scope note.

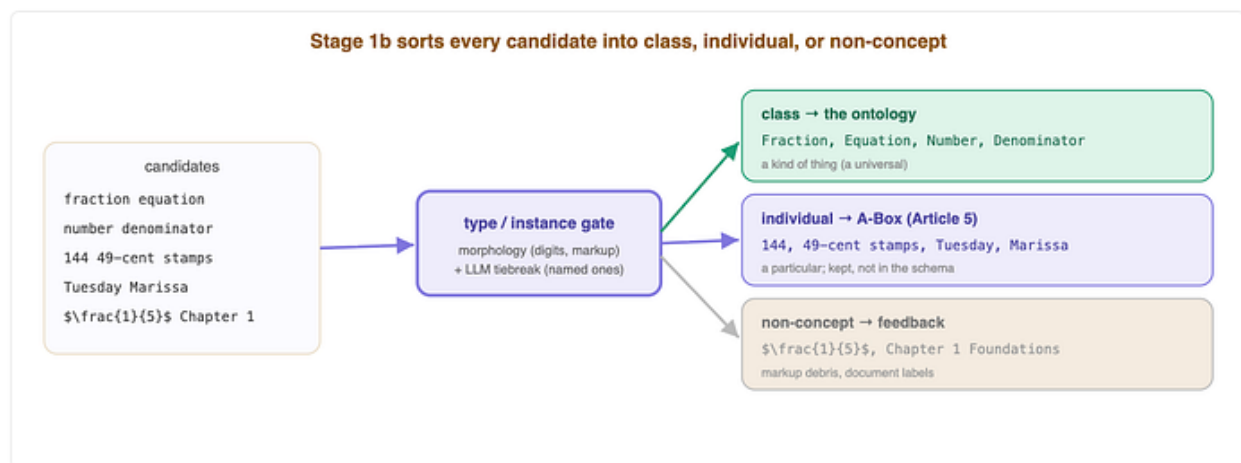


Figure 3. An algebra textbook example: The type/instance gate. Morphology routes the clear cases (a digit marks an individual, markup marks a non-concept), and an LLM tiebreak catches the named individuals that have no numeric tell. Only the classes flow into the ontology; particulars are routed to the A-Box (the knowledge graph), and debris to feedback.

7. Stage 2: Name the vocabulary

Purpose. Turn the noisy mention bag into a clean, flat vocabulary of named elements. This is the term to concept transition: many surface strings collapse into one named thing. The output is a flat set of concept names and relation names, each carrying its cluster of lexicalizations and its pooled provenance.

The mechanism is group, then select, then name. **Group** synonymous mentions by meaning, using context-enriched embeddings rather than string similarity: `IC` and `incident commander` are one concept because the sentences around them coincide. A **lexical guard** keeps the embeddings honest in a homogeneous domain (two terms merge only if they also share a content word or one is the acronym of the

other), so co-hyponyms like `numerator` and `denominator` stay separate. **Select** minimally, dropping only the obvious non-concepts. **Name** each survivor with one canonical label (`IncidentCommander`), keeping every surface as a recorded alternate label.

8. Stage 2b: Rank by salience

Purpose. Keep the salient core of the vocabulary and park the long tail. After gating, the vocabulary is on-topic but still long, and a domain's ontology is its central concepts, not every common noun an author used once. Each concept is scored for centrality and the vocabulary is split into a kept core and a parked tail (feedback, not deletion).

The score combines four signals already in the pipeline's artifacts, so it is explainable: a **scope match** (the concept is one the author defined or named in Stage 0, the strongest signal), **frequency**, **spread** across the document, and **grounding** by a hallucination-free extractor. A concept is kept if it is scope-matched or frequent; everything else is parked with a reason, and the kept core is what the structure stage commits to.

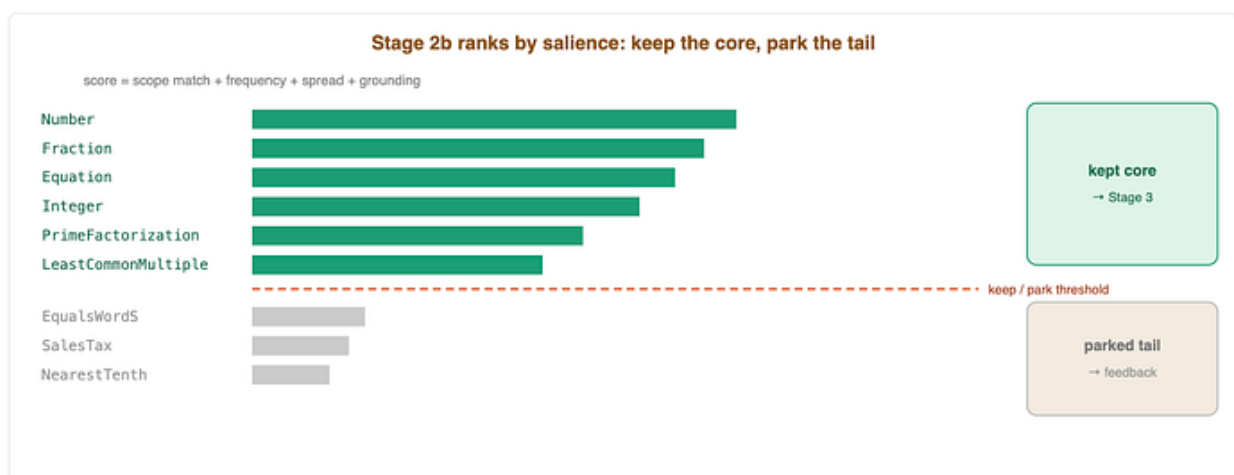


Figure 4. An algebra textbook example: Salience scoring. Each concept gets a score from four signals already in the pipeline (scope match is the strongest). Concepts above the keep/park line form the focused core the structure stage commits to; the long tail is parked as feedback, not deleted, and a later pass can promote it once it earns more mentions.

9. Stage 3: Synthesize structure

Purpose. The first stage that commits to structure. It wires the salient vocabulary into a shape: an `rdfs:subClassOf` DAG with a small number of coined abstract parents, a domain and range on every relationship, axioms, and a consistency report from a reasoner.

Earlier stages keep the LLM on a tight leash because it is least trustworthy when it free-guesses. Structure is where its strength (organizing a whole set into a coherent hierarchy, assigning sensible domain and range) is worth using, so here the LLM **synthesizes** the ontology, but **fenced on both ends**. It is given only the salient candidates and the Stage 0 scope, and it is told to build the taxonomy from the candidate labels, to omit duplicates and procedural terms, and to coin only a few clearly-needed abstract parents (which it must flag). Then every label it produced is **grounded** back to a candidate, inheriting real provenance; anything it introduces that is not a candidate (and not a flagged coined parent) is kept but marked, so review must corpus-check it. This is the same instinct as the terms-only leash, one level up: the model may organize, but it may not invent.

Anchor before you coin

Synthesis honors the rule from Section 2. Each class attaches to an existing class where one fits (a more general candidate, or, transitively, a verified parent SCIMA aligns to) before a new parent is coined. A **coined parent** is a hypothesis, tagged and justified by its children: where a family clearly exists with no word to name it (the three responder units), the pipeline coins `ResponderUnit` over them. Naming a coined parent well is deferred to review, where the LLM proposes a name from the children. True orphans are left alone, flagged, not force-parented.

Relations get a real domain and range

Because the LLM assigns domain and range over the just-built class set, relations come out clean: `dispatchedTo` with domain `ResponderUnit` and range `Incident`, `isAFactorOf` with domain `Factor` and range `Number`. This is a marked improvement over inferring domain and range from raw co-occurrence. Finally the axioms are emitted and a lightweight reasoner runs: it breaks any cycle in the DAG and relaxes the most suspect axiom (a coined disjointness before an extracted `subClassOf`) rather than dropping a class, and reports the result.

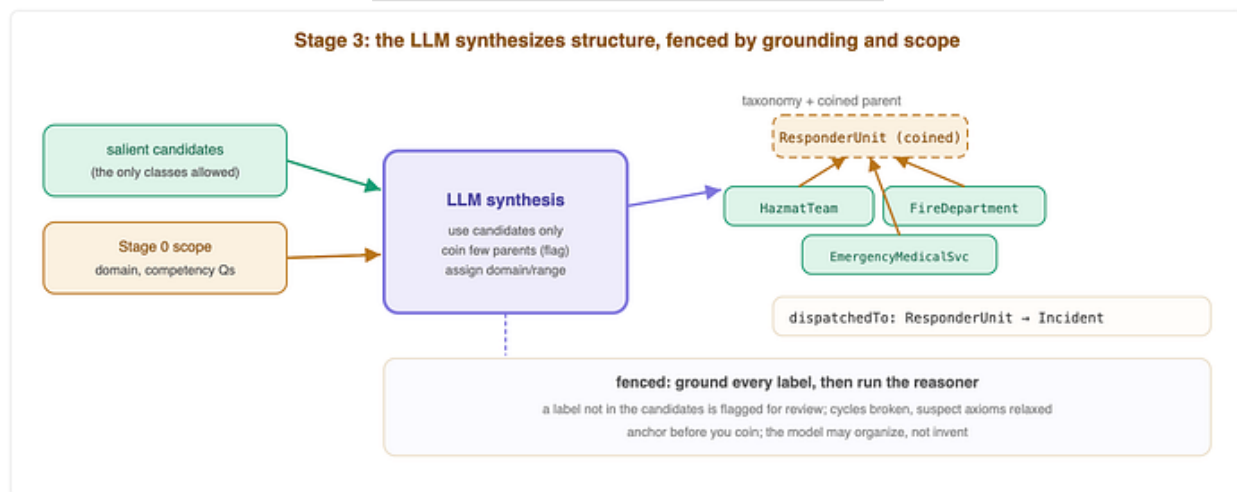


Figure 5. Structure synthesis. The LLM sees only the salient candidates and the scope, organizes them into a taxonomy (coining a flagged parent such as `ResponderUnit` where a family has no name) and assigns a domain and range to each relation. Every label is then grounded back to a candidate, anything introduced is flagged, and a reasoner closes the stage.

10. Stage 4: Agentic RITE review

Purpose. The trust gate. Everything upstream optimized for recall; this stage buys back precision. An AI agent drives the loop by calling tools rather than guessing, and a human ratifies the escalations. The input is the proposed structure with a dossier per element (provenance, confidence, coined-or-extracted, flags, reasoner report); the output is the admitted ontology plus a feedback set.

The agent's authority is bounded so a human is never surprised by what it admitted. It **auto-accepts** high-confidence elements grounded by their kind and consistent; **auto-rejects** clear hallucinations (an extracted concept with no corpus support at all); and **escalates** the genuinely ambiguous (a flagged introduction, a multi-parent node, a relationship with no resolvable domain and range) to the human. It tests each element by kind: an extracted concept by corpus grounding, a coined parent by having at least two grounded children (one child demotes it to a plain class), a relationship by predicate grounding plus a sane domain and range, an axiom by the reasoner.

Verb	What the agent does
Refine	Cheap fixes first: merge near-duplicate labels, demote an unjustified coined parent, park an orphan, and name a coined family (the deferred naming lands here). Recorded, human-approvable edits.
Inspect	Read each dossier. Coined, single-source, low-confidence, and flagged items get the hard look; a class with many mentions does not.
Test	Apply the test for the element's kind; repair inconsistency with the reasoner, re-running it after every edit.
Extend	Commit survivors. Park orphans under a domain top, flagged. Keep rejected and unconnected items in the feedback set for the next pass, never deleted.

The pipeline is a loop, not a waterfall. A Refine edit is an earlier stage run again (a re-parent is Stage 3, a rename or merge is Stage 2), and the feedback set (rejected, parked, unconnected) plus the admitted classes seed the next pass over more documents, so each pass starts from a richer schema.

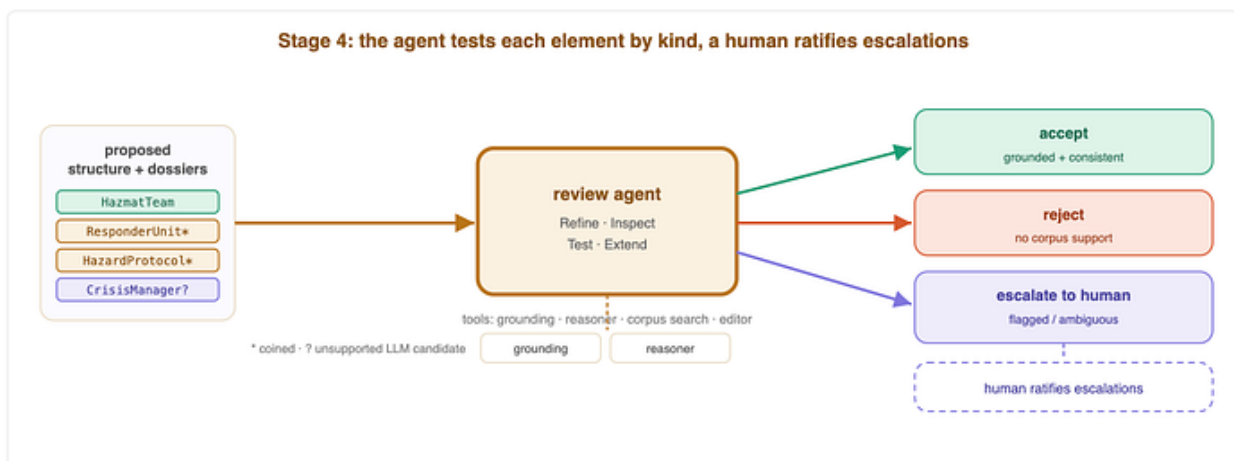


Figure 6. The agentic RITE review. The agent reads each dossier, tests it by kind with real tools, and routes it to one of three outcomes: it acts alone on the clear cases (accept the grounded, reject the unsupported) and escalates only the genuinely ambiguous, which a human ratifies. Here `ResponderUnit` is accepted, `HazardProtocol` is demoted, and `CrisisManager` is rejected.

11. SCIMA example: extracting v0.6 from the procedures corpus

Run the whole pipeline on a real SCIMA input. The city's emergency-management office ships a procedures handbook; one paragraph is the corpus for this pass.

When a hazardous-material spill is reported, the incident commander (IC) establishes a command post and designates an evacuation zone. A hazmat team is dispatched to contain the spill, coordinating with the fire department and emergency medical services. All responders must follow the hazardous-material protocol and wear PPE before entering the hot zone.

 literal span (cheap extractor)  LLM candidate (tagged, gets a harder look)

Scope and surface. Stage 0 reads the handbook's structure, states the domain (urban emergency response), lists competency questions ("which unit is dispatched to which incident?"), and drops the handbook's drill exercises and worked checklists so their specific case numbers never enter. Stage 1 unions the literal spans with the LLM candidates and dedups.

Sort, name, rank. Stage 1b sorts out the particulars: a specific spill case id, a timestamp, the responding officer's name go to the A-Box, not the ontology. Stage 2 groups IC with incident commander and names the survivors. The LLM also proposed CrisisManager, a plausible role that appears nowhere in the text; it survives tagged and unsupported, as recall-first requires. Stage 2b keeps the salient responder and incident concepts and parks one-off mentions.

Synthesize and review. Stage 3 synthesizes the structure: it attaches IncidentCommander under the existing Agent class and HazMatSpill under the existing Incident class (anchor before coin), coins ResponderUnit over the three responder units, and assigns dispatchedTo the domain ResponderUnit and range Incident. Stage 4 then tests by kind: ResponderUnit has three grounded children and is **accepted**; a coined parent over the single protocol has one child, fails its test, and is **demoted** to the plain class HazardousMaterialProtocol; CrisisManager has no corpus grounding and is **rejected**; EvacuationZone is admitted but parked under a domain top and flagged for re-parenting. The survivors are admitted as v0.6.

```

# SCIMA-OWL v0.6: emergency-response vocabulary extracted from the procedures corpus
(delta over v0.5)
@prefix scima: <http://scima.city/ontology#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix prov: <http://www.w3.org/ns/prov#> .

# --- Extracted classes (each links to the corpus span it came from) ---
scima:IncidentCommander a owl:Class ;
    rdfs:subClassOf scima:Agent ;                # anchored to the Agent class from v0.5
    rdfs:label "Incident Commander" ;
    skos:altLabel "IC" ;
    prov:wasDerivedFrom <corpus/emergency_procedures.txt#span_0142> .

scima:HazMatSpill a owl:Class ;
    rdfs:subClassOf scima:Incident ;                # anchored to the Incident class from
v0.2
    rdfs:label "Hazardous Material Spill" .

scima:ResponderUnit a owl:Class ;                # COINED: no single span; justified by
3 grounded children
    rdfs:label "Responder Unit" .
scima:HazmatTeam a owl:Class ; rdfs:subClassOf scima:ResponderUnit ;
    rdfs:label "Hazmat Team" .
scima:FireDepartment a owl:Class ; rdfs:subClassOf scima:ResponderUnit ;
    rdfs:label "Fire Department" .
scima:EmergencyMedicalService a owl:Class ; rdfs:subClassOf scima:ResponderUnit ;
    rdfs:label "Emergency Medical Service" .

scima:HazardousMaterialProtocol a owl:Class ; rdfs:label "Hazardous Material Protocol" .
# NB: a coined parent over this single protocol was DEMOTED in Stage 4 (only one grounded
child).

scima:EvacuationZone a owl:Class ; rdfs:label "Evacuation Zone" .
# NB: orphan; parked under a domain top and flagged for the next extraction pass.

# --- Relationships (domain and range assigned over the synthesized class set) ---
scima:commands a owl:ObjectProperty ; rdfs:domain scima:IncidentCommander ; rdfs:ra
nge scima:Incident .
scima:dispatchedTo a owl:ObjectProperty ; rdfs:domain scima:ResponderUnit ; rdfs:ra
nge scima:Incident .
scima:designates a owl:ObjectProperty ; rdfs:domain scima:IncidentCommander ; rdfs:ra
nge scima:EvacuationZone .
scima:followsProtocol a owl:ObjectProperty ; rdfs:domain scima:ResponderUnit ; rdfs:ra
nge scima:HazardousMaterialProtocol .

# --- Axioms emitted in Stage 3, checked by the reasoner before admission ---
scima:HazmatTeam owl:disjointWith scima:FireDepartment .
scima:HazmatTeam owl:disjointWith scima:EmergencyMedicalService .
scima:FireDepartment owl:disjointWith scima:EmergencyMedicalService .

```

```
# --- Routed out of the ontology ---
# scima:CrisisManager -- LLM candidate, no corpus grounding, auto-rejected (feedback set).
# specific spill ids, timestamps, responder names -- individuals, routed to the A-Box (Article 5).
```

SCIMA-OWL v0.6 admitted delta. The two reused parents (Agent , Incident) come from earlier versions; particulars were sorted out in Stage 1b; CrisisManager was rejected in review.

Anchoring v0.6 to verified ontologies

The extracted classes do not float free. Each attaches to a SCIMA parent, and SCIMA's top classes align to the expert-built foundations from Section 2, so every learned class inherits a verified foundation. The orphan finds a home too:

EvacuationZone is plainly a spatial feature, so it anchors to geo:Feature .

```
# Alignment layer (illustrative; NOT counted in the v0.6 totals).
@prefix scima: <http://scima.city/ontology#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix prov: <http://www.w3.org/ns/prov#> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix sosa: <http://www.w3.org/ns/sosa/> .
@prefix geo: <http://www.opengis.net/ont/geosparql#> .
@prefix emp: <https://w3id.org/empathi/> .

scima:Agent      rdfs:subClassOf prov:Agent , foaf:Agent .
scima:SensorDevice rdfs:subClassOf sosa:Sensor .
scima:Incident    rdfs:subClassOf emp:Hazard .
scima:ResponderUnit rdfs:subClassOf foaf:Organization .
scima:EvacuationZone rdfs:subClassOf geo:Feature .
```

Notice what the pipeline refused to do. It never admitted a class it could not trace to the text (CrisisManager is gone). It never let an instance into the schema (specific spills and times went to the A-Box). It never let a coined hypothesis through on faith (the lone protocol parent was demoted). And it never silently dropped a term it was unsure about (EvacuationZone is parked, not deleted). Recall first, provenance throughout, grounding by kind, universals not particulars, scope first.

Key takeaways

- Ontologies are built two complementary ways: **manually by experts** (the foundation) and **(semi-)automatically by machine** (the domain layer beneath it).
- An ontology is a **conceptualization**: it holds the **universals** of a domain, not the **individual facts** a text mentions. Separating the two (T-Box vs A-Box) is the main thing an extractor works.
- **Anchor before you coin**: reuse a verified upper or domain ontology (BFO, DOLCE, SUMO, PROV-O, SOSA/SSN, GeoSPARQL, Empathi) and attach to existing classes before coining new ones.
- A pipeline with **seven stages is developed**: Scope (frame and drop exercises), Surface (max recall), Sort by kind (type/instance gate), Name (term to concept), Rank by salience (keep the core), Synthesize structure (grounded LLM), and Review (the trust gate).
- It is a **loop, not a waterfall**: any later stage can send work back, and admitted classes plus the feedback set seed the next pass.
- Five invariants run the whole length: **recall-first, provenance everywhere, grounding by kind, universals not particulars, and scope before you extract**.
- The LLM is used where it is strong (scoping, sorting, naming, synthesizing structure and domain/range) but always **fenced by grounding**: it may organize, not invent.
- **The last Stage 4** is an agentic **RITE** loop (Refine, Inspect, Test, Extend): auto-accept the grounded, auto-reject hallucinations, escalate the ambiguous, test each element by its kind, and name the coined parents.

Further reading

- Marti A. Hearst. 1992. Automatic Acquisition of Hyponyms from Large Text Corpora. In COLING 1992 Volume 2: The 14th International Conference on Computational Linguistics.
- Cimiano, P. (2010). Ontology Learning and Population from Text: Algorithms, Evaluation and Applications. Springer. ISBN:978-1-4419-4032-2.
- Babaei Giglou, H., D'Souza, J., & Auer, S. (2023, October). LLMs4OL: Large language models for ontology learning. In International semantic web conference (pp. 408-427). Cham: Springer Nature Switzerland.
- Guarino, N., & Welty, C. (2002). Evaluating ontological decisions with OntoClean. Communications of the ACM, 45(2), 61-65.
- Guarino, N., Oberle, D., & Staab, S. (2009). What is an ontology? In Handbook on ontologies (pp. 1-17). Berlin, Heidelberg: Springer Berlin Heidelberg.

- Arp, R., Smith, B., & Spear, A. D. (2015). Building ontologies with basic formal ontology. Mit Press.
- Gruninger, M. (1995). Methodology for the design and evaluation of ontologies. In Proc. IJCAI'95, Workshop on Basic Ontological Issues in Knowledge Sharing.
- Hendler, J., Gandon, F., & Allemang, D. (2020). Semantic web for the working ontologist: Effective modeling for linked data, RDFS, and OWL. Morgan & Claypool.

[AI_agent_applications/ontology_KG_extraction_skills/](#)

[ontology_extraction_skills at main ·...](#) (https://github.com/anyuanay/AI_agent_applications/tree/main/ontology_KG_extraction_skills/ontology_extraction_skills)

Contribute to anyuanay/AI_agent_applications development by creating an account

on GitHub.

[Table of Content](#) (<https://freedium-mirror.cfd/https://medium.com/@anyuanay/ontology-and-knowledge-graphs-and-ai-agents-ce172be02ea1>)

If you found this helpful, clap 🖐️ to help others discover it, and follow for more!